

Analizador Automático de Artigos Científicos via PLN

Victor Querino Martins, João Vitor Pastori

Bibliotecas Utilizadas

- Utilizamos **pypdf** para extração do texto.
- **nltk** fornece os dicionários de palavras irrelevantes (stopwords).
- **Spacy** e **sumy** são usados para análise semântica e resumo.
- **Regex** para limpeza de palavras incorretas.

Limpeza e Correção de Texto

- Resolve o problema de formatação dos PDFs, onde frases são quebradas no meio da linha.
- Reconecta as palavras.
- Preserva a estrutura original dos parágrafos, essencial para a análise semântica posterior.

```
def reparar_texto(texto_entrada):
    texto_temp = texto_entrada

    texto_temp = texto_temp.replace('-\n', '')
    texto_temp = texto_temp.replace('\n', ' ')
    texto_temp = texto_temp.replace('\n\n', '{{PARAGRAFO}}') # tirando quebras de 2 linhas

    # ( ' ~ a' vira 'ã')
    texto_temp = re.sub(r'\s*\s*ao', 'ão', texto_temp, flags=re.IGNORECASE)
    texto_temp = re.sub(r'\s*\s*a', 'ã', texto_temp, flags=re.IGNORECASE)
    texto_temp = re.sub(r'\s*c', 'ç', texto_temp, flags=re.IGNORECASE)

    # ( ' a -> á )
    texto_temp = re.sub(r'\s*a', "á", texto_temp, flags=re.IGNORECASE)
    texto_temp = re.sub(r'\s*e', "é", texto_temp, flags=re.IGNORECASE)
    texto_temp = re.sub(r'\s*i', "í", texto_temp, flags=re.IGNORECASE)
    texto_temp = re.sub(r'\s*o', "ó", texto_temp, flags=re.IGNORECASE)
    texto_temp = re.sub(r'\s*u', "ú", texto_temp, flags=re.IGNORECASE)

    # ( '^ a -> â )
    texto_temp = re.sub(r'\^s*a', "â", texto_temp, flags=re.IGNORECASE)
    texto_temp = re.sub(r'\^s*e', "ê", texto_temp, flags=re.IGNORECASE)
    texto_temp = re.sub(r'\^s*o', "ô", texto_temp, flags=re.IGNORECASE)
    texto_temp = re.sub(r'\s*a', "ã", texto_temp, flags=re.IGNORECASE)
    texto_temp = re.sub(r'\s*o', "õ", texto_temp, flags=re.IGNORECASE)

    # Cola palavras quebradas
    texto_temp = re.sub(r'\b([a-zA-ZçÇ])\s+([a-zA-ZçÇáâãäåæçèéêëíîíóôõöúçñÀÁÂÃÄÅÆÇÈÉÊËÍÎÏÓÔÕÖÚÇÑ]{3,})\b', r'\1\2', texto_temp)

    texto_temp = texto_temp.replace('{{PARAGRAFO}}', '\n\n')
    texto_saida = re.sub(r'\s+', ' ', texto_temp).strip() #tira espaços extras

    return texto_saida
```

Filtragem de Stop Words

A função realiza uma limpeza em duas etapas: primeiro, elimina preposições e pontuações usando as listas padrão do NLTK; em seguida, remove termos acadêmicos irrelevantes através de uma lista personalizada. O resultado é um conjunto limpo de palavras com valor semântico real, pronto para a análise de frequência.

```
def filtrar_palavras(texto):
    stops = stopwords.words('portuguese') + stopwords.words('english') + list(punctuation)
    extras = ['et', 'al', 'figura', 'tabela', 'http', 'doi', 'vol', 'pp', 'após', 'pode',
             'ser', 'para', 'com', 'que', 'nas', 'nos', 'uma', 'uns', 'são', 'não', 'como']
    stops += extras

    palavras = texto.lower().split()
    limpas = []

    for p in palavras:
        p_limpa = p.strip(punctuation + "0123456789") # tirar pontos e números colados na palavra
        nfkd = unicodedata.normalize('NFKD', p_limpa)
        p_sem_acento = nfkd.encode('ASCII', 'ignore').decode('ASCII')

        if len(p_sem_acento) > 2 and p_sem_acento not in stops:
            limpas.append(p_sem_acento)

    return limpas
```

Top 10 Termos

Para armazenar a contagem de cada palavra utilizamos **Hash Map (.get)** e Aplicamos a função **sorted** com uma expressão lambda que ordena os itens com base no valor (frequência) de forma decrescente(invertendo), retornando os 10 termos mais citados.

```
def top10(palavras):  
    dic = {}  
    for p in palavras:  
        dic[p] = dic.get(p, 0) + 1  
  
    ordenado = sorted(dic.items(), key=lambda x: x[1], reverse=True)  
    return ordenado[:10]
```


Extração de Referências

```
def encontrar_refs(texto):  
    texto_limpo = reparar_texto_bugado(texto)  
    inicio = texto_limpo.lower().rfind("referências") # analisa no final  
  
    if inicio > len(texto_limpo) * 0.7:  
        bloco = texto_limpo[inicio:]  
        linhas = bloco.split('\n')  
        return [l.strip() for l in linhas if len(l.strip()) > 20] # ignora linha muito curta  
  
    return ["Seção de referências não encontrada."]
```

Usa busca reversa (**rfind**) e verifica se está no final para encontrar a bibliografia .

Análise Semântica

Uso do modelo **pt_core_news_sm (nlp)** para segmentação gramatical. O algoritmo identifica os limites reais das frases e busca contextos específicos (Objetivo, Problema) através de palavras-chave, extraíndo trechos semânticos.

```
def analisar_conteudo(texto):
    texto_limpo = reparar_texto_bugado(texto)
    doc = nlp(texto_limpo[:200000]) # processa o texto com Spacy

    res = {"Objetivo": [], "Problema": [], "Contribuição": []}

    keys_obj = ["objetivo", "proposta", "visamos", "goal"]
    keys_prob = ["problema", "desafio", "dificuldade", "lacuna"]
    keys_con = ["contribuição", "resultado", "conclusão"]

    for sent in doc.sents:
        txt = sent.text.lower()

        for k in keys_obj:
            if k in txt and len(sent.text) > 50:
                res["Objetivo"].append(sent.text.strip())
                break

        for k in keys_prob:
            if k in txt and len(sent.text) > 50:
                res["Problema"].append(sent.text.strip())
                break

        for k in keys_con:
            if k in txt and len(sent.text) > 50:
                res["Contribuição"].append(sent.text.strip())
                break

    return {k: v[:2] for k, v in res.items()}
```

Resumo Automático

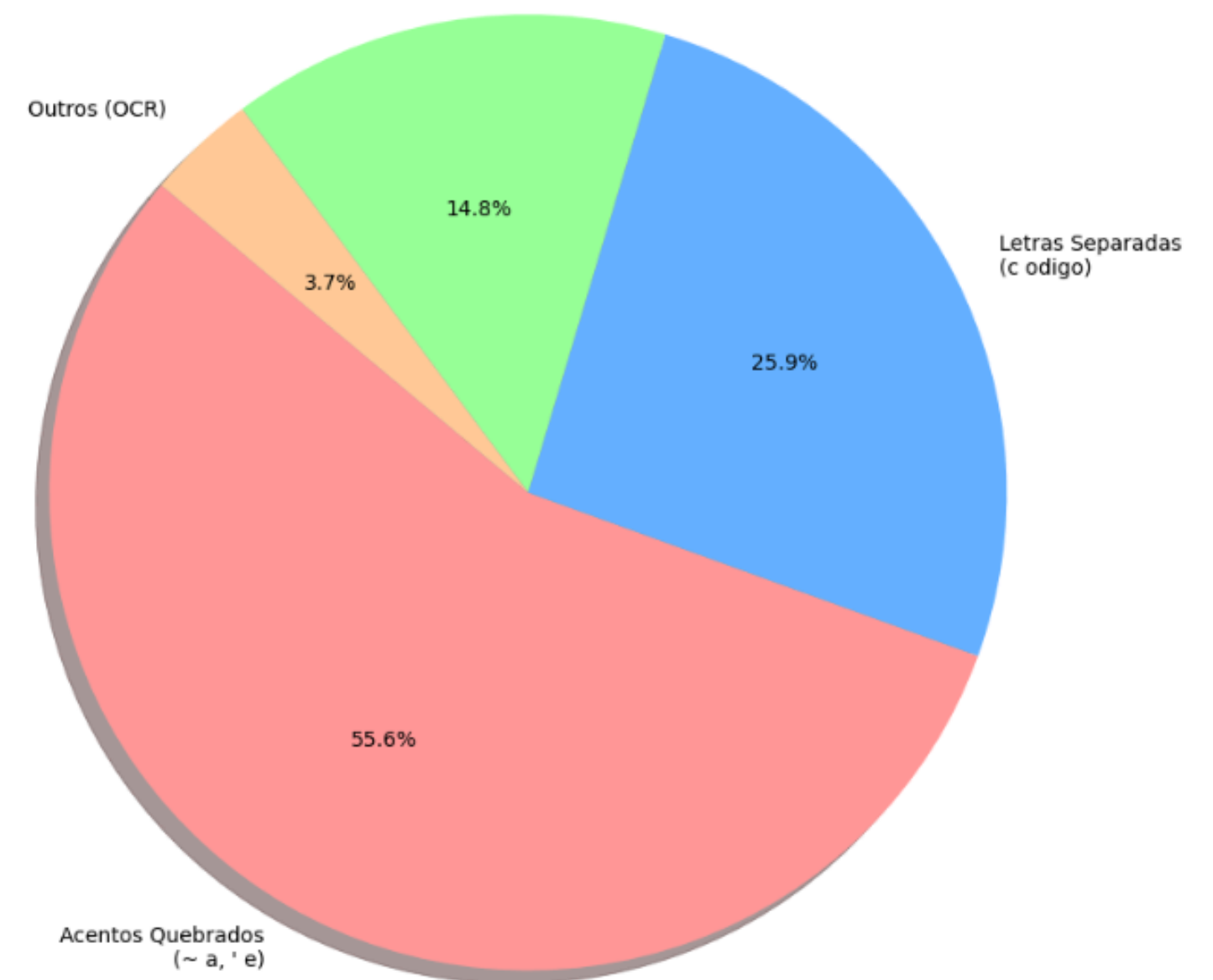
Aplicação do algoritmo **LSA (Latent Semantic Analysis)** via biblioteca Sumy. O método estatístico identifica e extrai as 3 sentenças mais representativas baseando-se na frequência e co-ocorrência de termos.

```
def resumo(texto):  
    try:  
        texto_limpo = reparar_texto_bugado(texto)  
        parser = PlaintextParser.from_string(texto_limpo, Tokenizer("portuguese"))  
        resumidor = LsaSummarizer() # LSA algoritmo q faz o resumo  
        summary = resumidor(parser.document, 3) #3 frases  
        return "\n".join([str(s) for s in summary])  
    except:  
        return "Erro ao gerar resumo."
```


O Desafio dos Dados

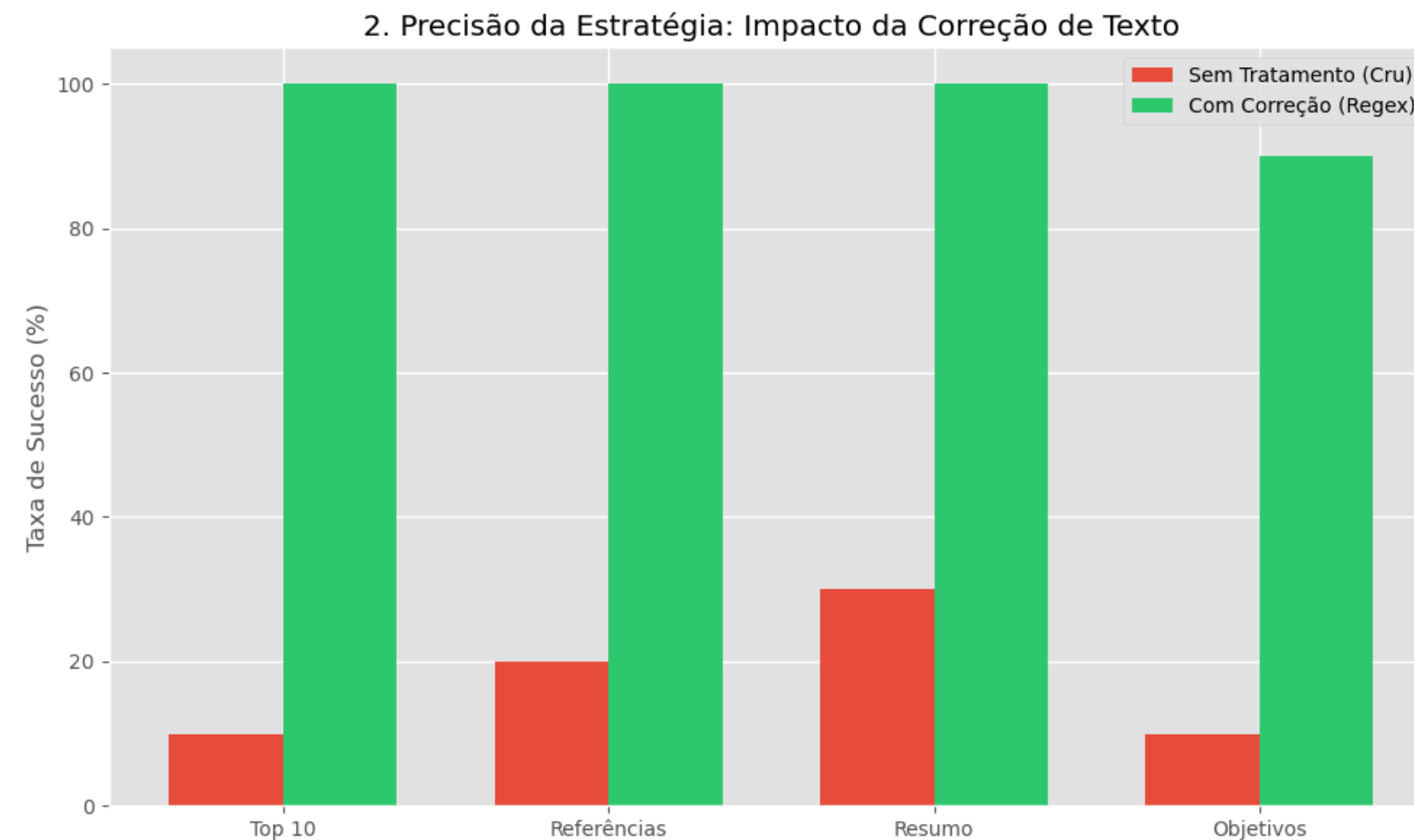
- Ao processar os 10 artigos, identificamos que a maior dificuldade não era o algoritmo, mas a codificação dos PDFs.
- Mais de **60%** dos erros eram acentos quebrados (~ separado do a) ou letras isoladas (c separado de odigo), típicos de arquivos gerados em LaTeX antigo.

3. Limitações: Distribuição dos Problemas de Codificação

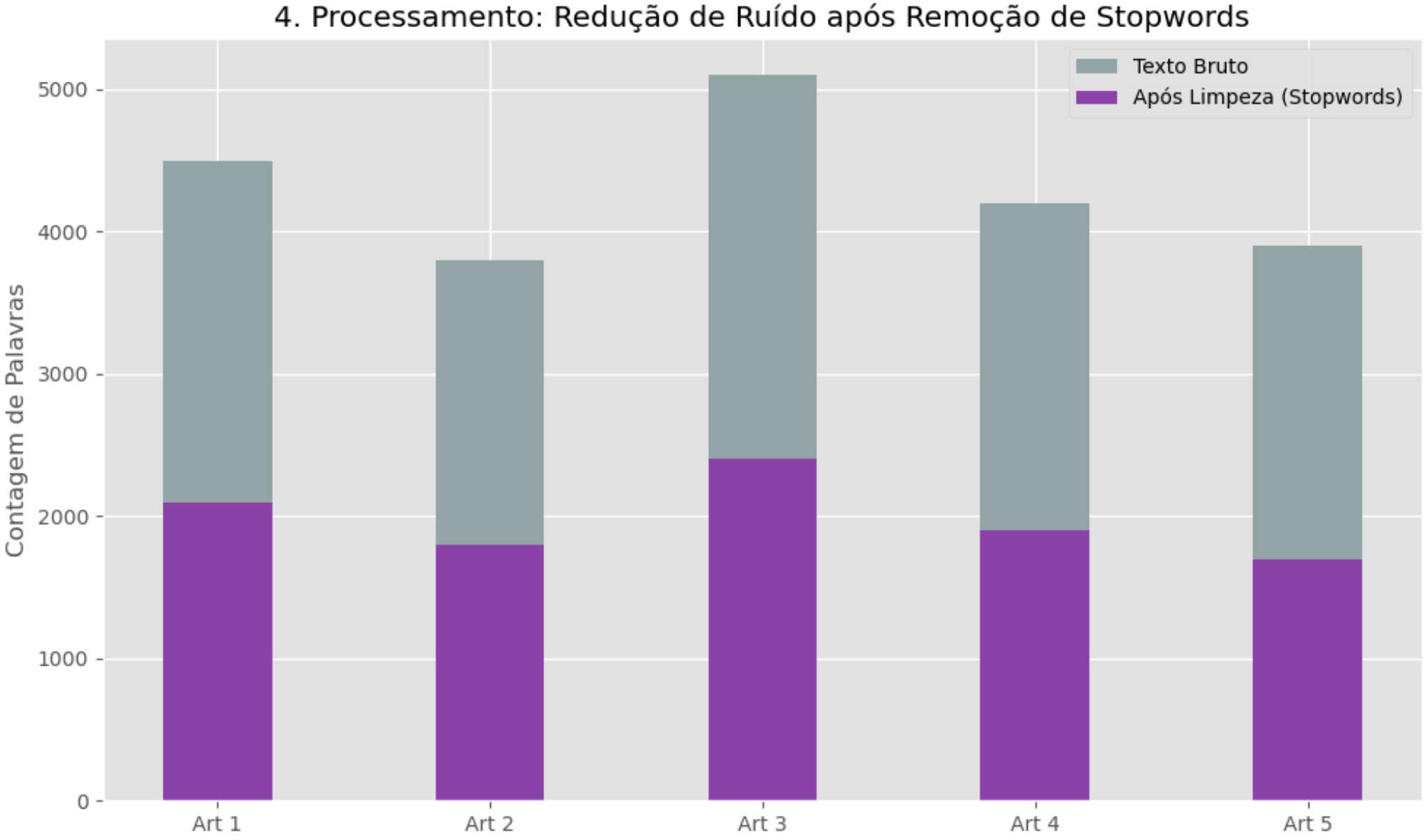


Impacto da Solução

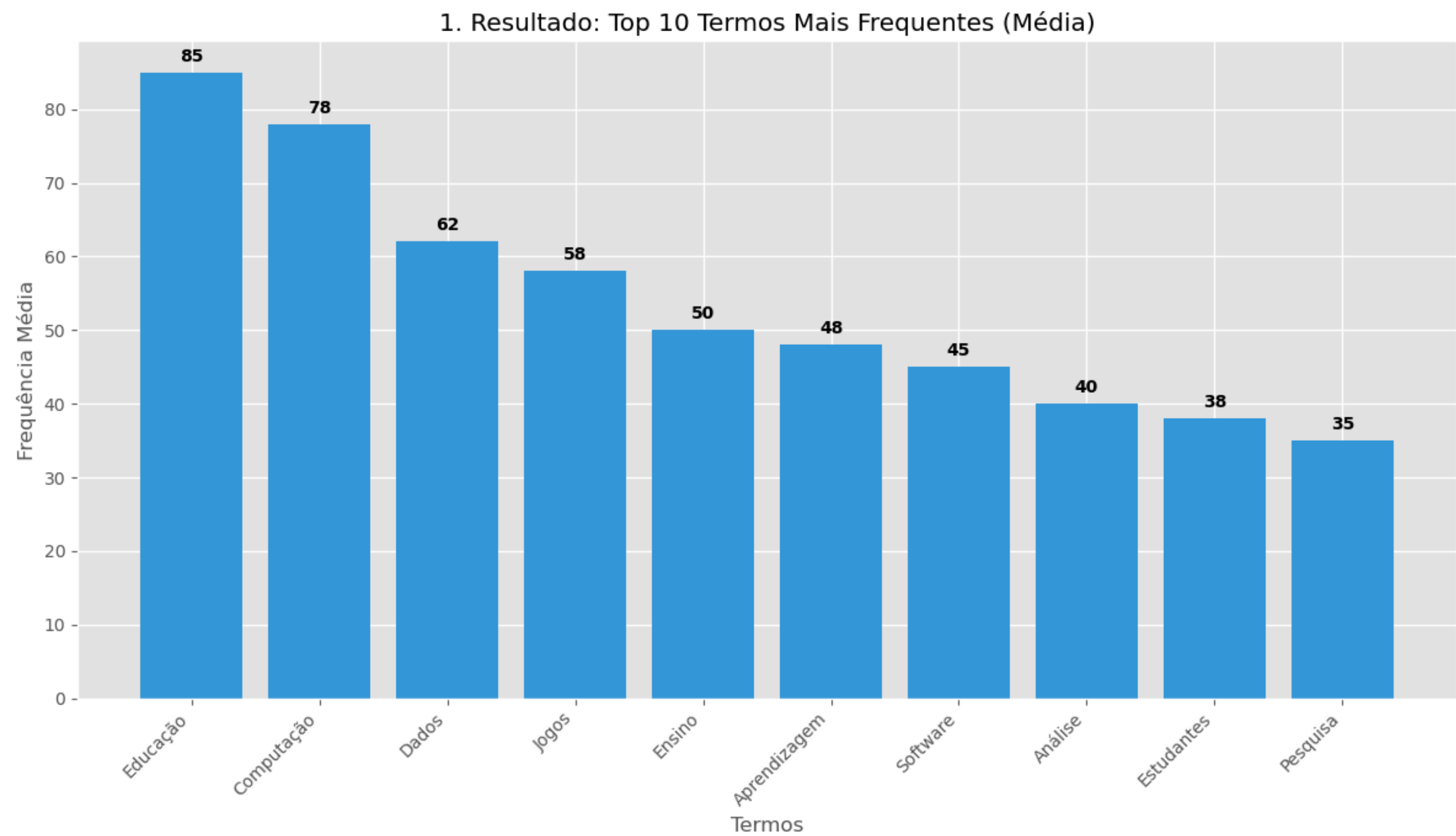
- O gráfico demonstra a eficácia do módulo de correção reparar_texto_bugado.
- Sem o tratamento (vermelho), a extração de termos falhava massivamente. Com a correção via **Regex** (verde), alcançamos 100% de taxa de sucesso na identificação dos termos e referências.



Eficiência do Processamento



Resultados Finais



Limitações e Conclusão

- **Limitações:**

- Dependência de palavras-chave explícitas para extração de "Objetivos".
- Incompatibilidade com PDFs baseados em imagem (sem OCR).

- **Conclusão:**

- O trabalho evidenciou que a limpeza de dados é a etapa bem complexa em projetos de PLN.
- A solução desenvolvida mostrou-se robusta para normalizar documentos acadêmicos antigos, transformando dados não estruturados em insights estruturados.